
Projet Intégrateur : Véhicule Autonome

Navigation par évitement d'obstacles, machine à états et intégration système (Option B)

Darix SAMANI SIEWE
Rapport de projet –Système a Microcontrolleurs

1^{er} juillet 2026

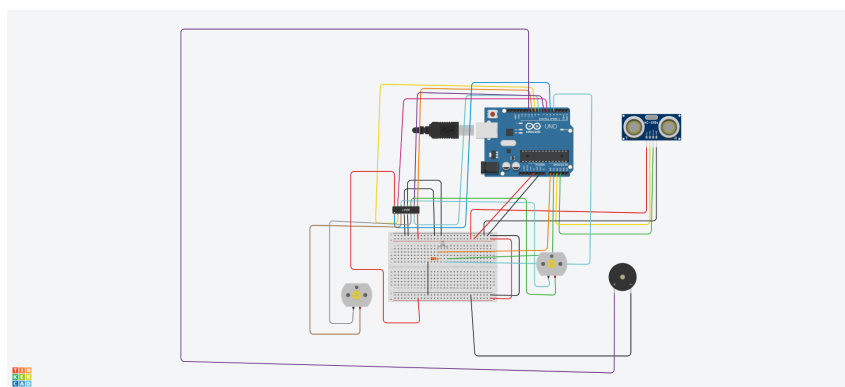


FIGURE 1 – Vue du montage du projet (capture Tinkercad ou photo réelle à insérer ici).

Table des matières

1	Introduction	2
2	Description du Projet	2
3	Composants Utilisés	2
3.1	Composants du cahier des charges (Option B – version complète)	2
3.2	Composants utilisés dans la simulation Tinkercad	2
4	Notions Électroniques Clés	3
5	Schéma du Montage	4
6	Code Arduino	4
7	Analyse et Discussion du Code	7
8	Ressources Complémentaires	8
9	Conclusion	8

1 Introduction

Ce rapport présente le **projet intégrateur** du module, conçu comme l'aboutissement du parcours pédagogique en mobilisant l'ensemble du kit électronique étudié. Le cahier des charges proposait deux options de réalisation : un **bras robotique** à 3 degrés de liberté piloté par joystick (Option A), ou un **véhicule autonome** capable d'éviter les obstacles (Option B). C'est cette seconde option qui a été retenue et développée dans ce rapport.

Le véhicule repose sur un **capteur ultrason HC-SR04** pour la détection d'obstacles, un **pont en H L298N** pour le pilotage de deux moteurs à courant continu, ainsi qu'une interface homme-machine simplifiée combinant une **LED RGB** de statut et un **buzzer** d'alarme. Le cahier des charges complet prévoyait également une commande sans fil par module **RF 433 MHz** et un affichage **OLED** de la télémétrie ; dans la version simulée sous Tinkercad, ces deux fonctions sont remplacées par une **liaison série** (commandes texte **START/STOP** depuis le moniteur série), qui joue un rôle fonctionnellement équivalent à la télécommande RF, et par l'affichage de la télémétrie directement dans le moniteur série en l'absence d'écran OLED.

2 Description du Projet

Le véhicule autonome fonctionne selon le principe suivant :

- au démarrage, le robot est à l'**arrêt** et attend une commande d'activation (LED bleue) ;
- une fois activé (commande **START**), il **avance** en continu (LED verte) tout en surveillant la distance à l'obstacle le plus proche via le capteur ultrason ;
- lorsqu'un obstacle est détecté sous le seuil critique (25 cm), le véhicule enchaîne automatiquement une **manœuvre d'évitement** : recul bref (LED rouge, bip grave), puis rotation sur lui-même (LED jaune, bip aigu), avant de reprendre sa progression ;
- une commande **STOP** permet à tout moment un **arrêt d'urgence**, ramenant le système à l'état initial.

L'ensemble du comportement est structuré autour d'une **machine à états finis** à quatre états (**ARRET**, **AVANCE**, **RECU**, **ROTATION**), gérée de façon **non bloquante** grâce à `millis()`, ce qui permet au système de rester réactif aux commandes série et aux mesures de distance tout au long de la manœuvre d'évitement.

Objectif pédagogique : En tant que projet intégrateur, ce véhicule combine la quasi-totalité des notions abordées dans les projets précédents : acquisition de capteur (ultrason), pilotage d'actionneurs de puissance (moteurs DC via pont en H), interface de statut (LED RGB, buzzer), communication (liaison série en lieu et place du RF433), et surtout une **architecture logicielle unifiée** sous forme de machine à états, illustrant l'intégration système au cœur du cahier des charges.

3 Composants Utilisés

3.1 Composants du cahier des charges (Option B – version complète)

3.2 Composants utilisés dans la simulation Tinkercad

Remarque : Le module **RF 433 MHz** et l'écran **OLED** ne sont pas disponibles dans la simulation Tinkercad. Le code fourni anticipe explicitement cette contrainte : la variable `robotActive`, commentée « Remplacera le signal RF433 », est pilotée via des commandes texte (**START/STOP**) envoyées sur le moniteur série, qui se substitue fonctionnellement à la

lightgray Composant	Rôle dans le projet
HC-SR04 (x2)	Détection de distance pour la navigation et l'évitement d'obstacles
L298N	Pont en H pilotant les deux moteurs DC en vitesse et en sens de rotation
Moteurs DC (x2)	Motorisation des roues gauche et droite du véhicule
RF 433 MHz	Réception de commandes sans fil (activation/désactivation à distance)
LED RGB	Indicateur visuel de l'état du véhicule (attente, avance, recul, rotation)
Buzzer	Signal sonore associé aux manœuvres d'évitement (alarme/avertissement)
Arduino Uno	Microcontrôleur assurant l'acquisition, la machine à états et le pilotage moteur

TABLE 1 – Composants prévus pour l'Option B (véhicule autonome) du cahier des charges initial.

lightgray Réf- rence	Quantité	Composant
U1	1	Arduino Uno R3
U2	1	Pont en H (H-Bridge Motor Driver, type L298N)
M1, M2	2	Moteurs à courant continu (roues gauche et droite)
DIST1	1	Capteur de distance ultrasonique (4 broches, type HC-SR04)
D10	1	LED RGB (statut du véhicule)
PIEZO1	1	Buzzer piézoélectrique (alarme sonore)
R5	1	Résistance 220 Ω

TABLE 2 – Composants effectivement câblés dans la simulation Tinkercad.

télécommande radio. De même, la télémétrie (distance mesurée, état courant) est affichée dans le moniteur série plutôt que sur un écran OLED. Cette substitution n'altère en rien la logique de navigation et d'évitement, strictement identique à la version cible.

4 Notions Électroniques Clés

- **Algorithme d'évitement d'obstacles** : la stratégie implémentée ici est de type *réactif* – dès qu'un obstacle est détecté sous le seuil critique, le véhicule exécute une séquence fixe (recul puis rotation) avant de reprendre sa marche, sans cartographie ni planification de trajectoire. Cette approche, simple à mettre en œuvre, constitue la base de stratégies plus élaborées (évitement avec scan multi-directionnel, calcul d'angle de fuite optimal).
- **Gestion d'énergie et calcul de la consommation totale du système** : un véhicule autonome combine des consommateurs très différents (moteurs DC pouvant tirer plusieurs centaines de mA chacun sous charge, électronique de commande à quelques dizaines de mA) ; le dimensionnement de l'alimentation (autonomie de batterie, courant de pointe au démarrage des moteurs) est un point de conception critique, généralement sous-estimé dans les premières versions d'un tel projet.
- **Temps réel et priorité des tâches (interruptions vs boucle principale)** : ce projet

illustre une approche **coopérative** de gestion du temps (boucle principale scrutée en continu, temporisation par `millis()`), par opposition à une approche par **interruptions** qui serait nécessaire si, par exemple, la réception RF433 ou la lecture d'un encodeur de roue devait être traitée sans délai, indépendamment de la durée d'exécution de la boucle principale.

- **Intégration système – interconnexion de tous les sous-systèmes** : la complexité propre à un projet intégrateur réside moins dans chaque sous-système pris isolément (capteur, moteurs, LED, buzzer) que dans leur **coordination cohérente** au sein d'une architecture unique, ici matérialisée par la machine à états `EtatRobot`, qui centralise la décision et synchronise l'ensemble des actionneurs et indicateurs en fonction d'un seul état courant.

5 Schéma du Montage

6 Code Arduino

Le programme implémente l'ensemble de la logique de navigation autonome : écoute des commandes série, mesure de distance, machine à états de navigation/évitement, pilotage différentiel des moteurs via le pont en H, et retour visuel/sonore par LED RGB et buzzer.

```

1 /**
2  * PROJET 10 : V HICULE AUTONOME - VITEMENT D'OBSTACLES
3  * Navigation, T l m trie S rie et Machine tats finis
4  */
5
6 // --- Broches HC-SR04 ---
7 const int pinTrig = A2;
8 const int pinEcho = A3;
9
10 // --- Broches L298N ---
11 const int ENA = 10; const int IN1 = 9; const int IN2 = 8; // Moteur Gauche
12 const int ENB = 5; const int IN3 = 7; const int IN4 = 6; // Moteur Droit
13
14 // --- Broches IHM (RGB & Buzzer) ---
15 const int pinLedRouge = A0;
16 const int pinLedVerte = A1;
17 const int pinLedBleue = 4;
18 const int pinBuzzer = 11;
19
20 // --- Param tres de Navigation ---
21 const int DISTANCE_CRITIQUE = 25; // cm
22 const int VITESSE_MAX = 200; // PWM 0-255
23 const int VITESSE_MANOEUVRE = 150;
24
25 // --- Machine tats ---
26 enum EtatRobot { ARRET, AVANCE, REcul, ROTATION };
27 EtatRobot etatActuel = ARRET;
28
29 bool robotActive = false; // Remplacera le signal RF433
30 unsigned long chronoManoeuvre = 0;
31
32 void setup() {
33     Serial.begin(9600);
34
35     pinMode(pinTrig, OUTPUT); pinMode(pinEcho, INPUT);
36     pinMode(ENA, OUTPUT); pinMode(IN1, OUTPUT); pinMode(IN2, OUTPUT);
37     pinMode(ENB, OUTPUT); pinMode(IN3, OUTPUT); pinMode(IN4, OUTPUT);
38
39     pinMode(pinLedRouge, OUTPUT); pinMode(pinLedVerte, OUTPUT);

```

```

40  pinMode(pinLedBleue, OUTPUT); pinMode(pinBuzzer, OUTPUT);
41
42  couleurRGB(0, 0, 1); // Bleu = En attente d'activation
43  stopperMoteurs();
44
45  Serial.println(F("SYSTEME INITIALISE."));
46  Serial.println(F("Tapez 'START' pour lancer la navigation, 'STOP' pour arreter
47  ."));
48
49  void loop() {
50      unsigned long tempsCourant = millis();
51
52      // 1.  coule du canal de commande (Simule la t l commande RF 433MHz)
53      if (Serial.available() > 0) {
54          String commande = Serial.readStringUntil('\n');
55          commande.trim();
56          if (commande == "START") {
57              robotActive = true;
58              etatActuel = AVANCE;
59              Serial.println(F(">>> MODE AUTONOME ENGAGE <<<"));
60          } else if (commande == "STOP") {
61              robotActive = false;
62              etatActuel = ARRET;
63              Serial.println(F(">>> ARRET D'URGENCE <<<"));
64          }
65      }
66
67      if (!robotActive) {
68          stopperMoteurs();
69          couleurRGB(0, 0, 1); // Bleu
70          return; // Coupe l'ex cution de la boucle ici
71      }
72
73      // 2. Acquisition Capteur
74      long distance = mesurerDistance();
75
76      // 3. Logique de Navigation (Machine tats )
77      switch (etatActuel) {
78
79          case AVANCE:
80              if (distance > 0 && distance < DISTANCE_CRITIQUE) {
81                  // Obstacle d tect ! On passe en manoeuvre d'urgence
82                  etatActuel = RECUL;
83                  chronoManoeuvre = tempsCourant;
84                  Serial.print(F("OBSTACLE a ")); Serial.print(distance); Serial.println(F(
85                  (" cm. Evitement en cours...")));
86              } else {
87                  avancer(VITESSE_MAX);
88                  couleurRGB(0, 1, 0); // Vert
89                  noTone(pinBuzzer);
90              }
91              break;
92
93          case RECUL:
94              // Recule pendant 600 ms
95              if (tempsCourant - chronoManoeuvre < 600) {
96                  reculer(VITESSE_MANOEUVRE);
97                  couleurRGB(1, 0, 0); // Rouge
98                  tone(pinBuzzer, 1000); // Bip de recul (camion)
99              } else {
100                  etatActuel = ROTATION;
101                  chronoManoeuvre = tempsCourant; // Reset chrono

```

```

101     }
102     break;
103
104     case ROTATION:
105         // Tourne sur lui-même pendant 400 ms
106         if (tempsCourant - chronoManoeuvre < 400) {
107             tournerDroite(VITESSE_MANOEUVRE);
108             couleurRGB(1, 1, 0); // Jaune (Rouge + Vert)
109             tone(pinBuzzer, 2000); // Bip aigu de rotation
110         } else {
111             etatActuel = AVANCE; // Fin de la manoeuvre, on repart
112         }
113         break;
114
115     case ARRET:
116         stopperMoteurs();
117         break;
118 }
119
120 delay(50); // Stabilité du CPU
121 }
122
123 // =====
124 // FONCTIONS DE CONTRÔLE MOTEURS (L298N)
125 // =====
126 void avancer(int vitesse) {
127     analogWrite(ENA, vitesse); digitalWrite(IN1, HIGH); digitalWrite(IN2, LOW);
128     analogWrite(ENB, vitesse); digitalWrite(IN3, HIGH); digitalWrite(IN4, LOW);
129 }
130 void reculer(int vitesse) {
131     analogWrite(ENA, vitesse); digitalWrite(IN1, LOW); digitalWrite(IN2, HIGH);
132     analogWrite(ENB, vitesse); digitalWrite(IN3, LOW); digitalWrite(IN4, HIGH);
133 }
134 void tournerDroite(int vitesse) {
135     analogWrite(ENA, vitesse); digitalWrite(IN1, HIGH); digitalWrite(IN2, LOW); //
136     // G avance
137     analogWrite(ENB, vitesse); digitalWrite(IN3, LOW); digitalWrite(IN4, HIGH); //
138     // D recule
139 }
140 void stopperMoteurs() {
141     analogWrite(ENA, 0); analogWrite(ENB, 0);
142     digitalWrite(IN1, LOW); digitalWrite(IN2, LOW);
143     digitalWrite(IN3, LOW); digitalWrite(IN4, LOW);
144 }
145
146 // =====
147 // FONCTION CAPTEUR & LED
148 // =====
149 long mesurerDistance() {
150     digitalWrite(pinTrig, LOW); delayMicroseconds(2);
151     digitalWrite(pinTrig, HIGH); delayMicroseconds(10);
152     digitalWrite(pinTrig, LOW);
153     long duree = pulseIn(pinEcho, HIGH, 30000);
154     if (duree == 0) return 999;
155     return duree * 0.034 / 2;
156 }
157
158 void couleurRGB(bool r, bool g, bool b) {
159     digitalWrite(pinLedRouge, r);
160     digitalWrite(pinLedVerte, g);
161     digitalWrite(pinLedBleue, b);
162 }

```

Listing 1 – Code source Arduino du véhicule autonome – évitement d'obstacles (simulation)

Tinkercad).

7 Analyse et Discussion du Code

Le code peut être décomposé en six blocs fonctionnels principaux :

1. **Déclarations et initialisation (setup)** : toutes les broches (capteur, pont en H, LED RGB, buzzer) sont configurées en entrée ou sortie selon leur rôle. L'état initial place la LED en bleu (attente) et arrête les moteurs via `stopperMoteurs()`, garantissant un démarrage sécurisé du véhicule, immobile tant qu'aucune commande d'activation n'a été reçue.
2. **Réception des commandes (simulation RF433 par liaison série)** : la boucle principale teste en premier lieu la disponibilité de données sur le port série. Une commande **START** active le robot et le place en état **AVANCE**; une commande **STOP** le désactive et le ramène à l'état **ARRET**. La fonction `trim()` appliquée à la chaîne reçue élimine les éventuels caractères de fin de ligne parasites, une précaution utile pour fiabiliser le parsing de commandes texte.
3. **Garde d'inactivité (if (!robotActive) ... return;)** : tant que le robot n'est pas activé, la fonction `loop()` s'interrompt prématurément après avoir arrêté les moteurs et affiché la LED bleue, évitant d'exécuter inutilement la logique de navigation. Cette structure de « garde » en début de boucle est une pratique efficace pour simplifier la lisibilité du code principal.
4. **Mesure de distance (mesurerDistance)** : la fonction utilise le couple `trig/echo` classique du HC-SR04 avec un *timeout* de 30 ms sur `pulseIn()`. En cas d'absence d'écho (capteur hors de portée), une valeur conventionnelle élevée (999) est retournée plutôt que zéro, évitant une fausse détection d'obstacle due à une mesure invalide – un détail de robustesse important.
5. **Machine à états de navigation (switch (etatActuel))** : chaque état encapsule à la fois le pilotage moteur, l'indication LED et le signal sonore associé. L'état **AVANCE** surveille en continu la distance et déclenche la transition vers **RECU** dès franchissement du seuil critique. Les états **RECU** et **ROTATION** utilisent chacun un chronomètre dédié (`chronoManoeuvre`) pour borner leur durée (600 ms puis 400 ms) avant de passer à l'état suivant, formant ainsi une **séquence d'évitement fixe et reproductible**.
6. **Fonctions de pilotage moteur (avancer, reculer, tournerDroite, stopperMoteurs)** : chaque fonction combine une consigne de vitesse (PWM via `analogWrite()` sur ENA/ENB) et un sens de rotation (combinaison logique IN1–IN4). La rotation sur place est obtenue en faisant avancer un moteur tout en reculant l'autre simultanément (rotation différentielle), une technique classique pour les véhicules à propulsion différentielle.

Points forts du code :

- Architecture en machine à états claire, où chaque état regroupe de façon cohérente pilotage moteur, retour visuel et retour sonore.
- Gestion robuste des mesures de distance invalides (retour de 999 en cas de *timeout*), évitant les fausses détections d'obstacle.
- Anticipation explicite de l'intégration RF433 via la variable `robotActive` et son commentaire dédié, facilitant le remplacement futur de la liaison série par le module radio réel.
- Séparation nette entre logique de décision (machine à états) et fonctions d'action (pilotage moteur), facilitant la maintenance et les tests unitaires de chaque sous-

système.

Pistes d'amélioration possibles :

- Remplacer le `delay(50)` final de la boucle par une gestion entièrement non bloquante, afin de garantir une réactivité maximale du système, notamment lors de l'intégration future du module RF433.
- Intégrer un second capteur ultrason (le cahier des charges en prévoit deux) pour permettre un évitement directionnel (choix de tourner à gauche ou à droite selon le côté dégagé), plutôt qu'une rotation systématique dans le même sens.
- Ajouter un affichage de télémétrie sur écran OLED (distance courante, état du robot, niveau de batterie estimé) en complément du moniteur série, conformément au cahier des charges complet.
- Étudier le remplacement de la séquence d'évitement à durée fixe (600 ms / 400 ms) par une boucle de réévaluation continue de la distance pendant la manœuvre, pour s'adapter dynamiquement à des obstacles mobiles ou à des espaces de manœuvre restreints.

8 Ressources Complémentaires

- **Vidéo** – *Arduino Robotic Arm : 3 DOF with Joystick Control (Complete)*
<https://www.youtube.com/watch?v=O2RX6yGAFqY>
- **Vidéo** – *Arduino Obstacle Avoiding Robot Car : Full Build Tutorial*
<https://www.youtube.com/watch?v=sanFuKNiVA>
- **Vidéo** – *Véhicule autonome Arduino complet avec L298N (francophone)*
<https://www.youtube.com/watch?v=gJhQbWVbFxU>

9 Conclusion

En tant que projet intégrateur, ce véhicule autonome démontre la capacité à combiner acquisition de capteur, pilotage d'actionneurs de puissance et interface de retour utilisateur au sein d'une architecture logicielle unique et cohérente. La machine à états centralisée constitue le point fort de cette implémentation, rendant le système à la fois lisible et facilement extensible – que ce soit pour intégrer le module RF433 prévu au cahier des charges, ajouter un second capteur de distance pour un évitement directionnel, ou encore basculer vers l'Option A (bras robotique) en réutilisant la même rigueur architecturale.

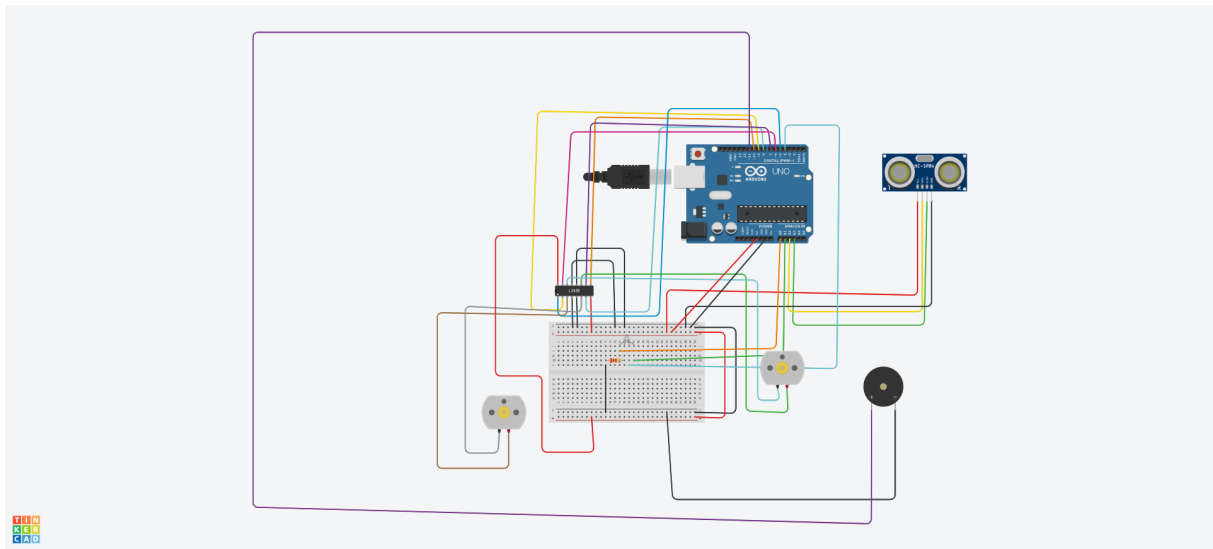


FIGURE 2 – Schéma de câblage du montage (Arduino Uno, pont en H L298N, 2 moteurs DC, capteur ultrason, LED RGB, buzzer).